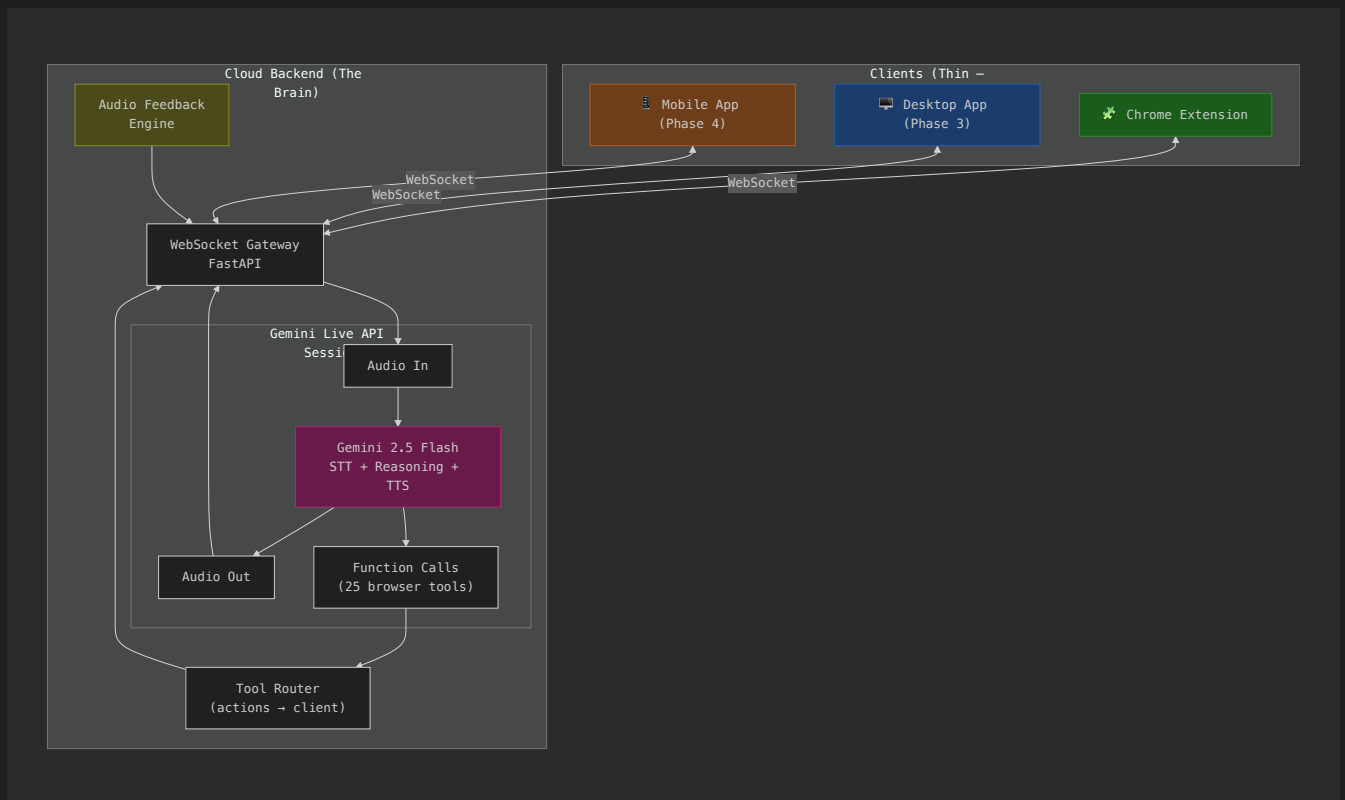


TernConnect – Final Implementation Plan

A cloud-deployed AI brain that gives blind users **full voice control** over web platforms – especially LMS systems. Deployable as Chrome extension (Phase 1), desktop app (Phase 3), and mobile app (Phase 4). Covers ~92% of blind-user LMS scenarios.

Architecture



Key Decisions

Decision	Rationale
Central deployed backend	One brain powers all clients. Update once, every platform improves.
Gemini Live API (not 3 separate APIs)	Single connection handles STT + reasoning + TTS. ~500ms vs ~2.5s. 60% cheaper.
Chrome extension as primary client	Controls user's real browser/DOM directly. Uses their existing sessions.
WebSocket protocol	Real-time bidirectional audio streaming.
Audio feedback engine	Blind users need continuous sound cues – silence = uncertainty.
Keyboard navigation fallback	When DOM clicks fail, Tab/Enter always works.
Cross-origin iframe injection	Payment forms and video embeds live in iframes – must reach them.

Project Structure

```
ternconnect/
├── backend/
│   ├── main.py                # FastAPI app, WebSocket endpoint
│   ├── config.py              # Env vars, model config, defaults
│   ├── requirements.txt        # Python dependencies
│   ├── .env.example            # API key template
│   ├── Dockerfile              # Cloud deployment
│   └── core/
│       ├── __init__.py
│       ├── session.py          # Per-client session: Gemini Live + state
│       ├── gemini_live.py      # Gemini Live API WebSocket client
│       ├── tools.py             # 25 tool/function definitions
│       ├── tool_executor.py     # Routes tool calls → client
│       └── prompts.py           # System prompt (10 rule sections)
│
│   ├── api/
│       ├── __init__.py
│       └── websocket_handler.py # Client WebSocket protocol handler
│
│   ├── audio/
│       ├── __init__.py
│       ├── feedback.py          # Audio feedback state machine
│       └── sounds/              # Feedback sound files
│           ├── chime_up.wav      # "I'm listening"
│           ├── chime_down.wav    # "Got it, processing"
│           ├── thinking.wav      # Looping pulse while AI reasons
│           ├── tick.wav          # Action executing
│           ├── success.wav       # Action succeeded
│           ├── error.wav         # Something went wrong
│           └── whoosh.wav        # Page loading/navigating
│
│   └── models/
│       ├── __init__.py
│       └── messages.py          # WebSocket message protocol types
│
├── extension/
│   ├── manifest.json            # MV3 manifest
│   └── background/
│       └── service-worker.js    # WebSocket + routing + shortcut + tab
│
├── monitor
│   └── content/
│       └── dom-controller.js    # Click, type, scroll, select, rich text,
│
├── drag-drop
│   └── page-reader.js           # Accessibility tree, math, PDF, session
│
├── monitor
│   └── video-handler.js         # Video detection + caption extraction
│   └── video-controller.js     # Play, pause, seek, speed, status
```

```

├── ── quiz-monitor.js      # Timer, questions, answers, drag-drop
├── ── accessibility.js    # Screen reader bridge + ARIA live regions
├── ── keyboard-navigator.js # Tab/arrow/enter fallback navigation
├── ── content.css         # Visual feedback styles
├── ── offscreen/
├── │   ├── offscreen.html  # Required for Web Audio in MV3
├── │   └── audio-handler.js # Mic capture + TTS + feedback sounds
├── ── popup/
├── │   ├── popup.html     # Status + settings (screen-reader
├── │   └── popup.js       # Connection status, settings logic
├── │   └── popup.css      # Accessible, high-contrast styles
├── ── shared/
├── │   ├── protocol.js    # Message types (mirrors backend)
├── │   └── config.js      # Backend URL, shortcut key, preferences
├── ── README.md

```

~26 files total.

[NEW] backend/main.py

FastAPI application with WebSocket endpoint.

- Mounts WebSocket at `/ws`
- Health check at `/health`
- CORS middleware for extension origin
- Lifespan handler for startup validation and shutdown cleanup

[NEW] backend/config.py

python

```
import os
from dotenv import load_dotenv

load_dotenv()

# API Keys
GEMINI_API_KEY: str = os.environ["GEMINI_API_KEY"]

# Models
GEMINI_MODEL: str = "gemini-2.5-flash"

# Server
HOST: str = os.getenv("HOST", "0.0.0.0")
PORT: int = int(os.getenv("PORT", "8000"))
CORS_ORIGINS: list[str] = ["chrome-extension://*", "http://localhost:*"]

# AI Behavior
CONVERSATION_HISTORY_MAX_TURNS: int = 20
ACTION_TIMEOUT_SECONDS: float = 10.0
MAX_RETRY_ATTEMPTS: int = 3

# Audio Feedback
FEEDBACK_SOUNDS_DIR: str = os.path.join(os.path.dirname(__file__), "audio", "sounds")
```

[NEW] backend/core/session.py

Per-client session that bridges the WebSocket client and a Gemini Live API session.

Responsibilities:

- Opens Gemini Live session with all 25 tools and the system prompt
- Forwards client mic audio → Gemini for speech understanding
- Forwards Gemini TTS audio → client for playback
- When Gemini calls a tool → routes to the correct client message type
- When client returns a tool result → forwards to Gemini
- Sends audio feedback sounds at state transitions
- Handles page load monitoring (client reports loading/loaded events)
- Handles tab changes (client reports new tabs from OAuth/popups)
- On error: sends error to Gemini so it can recover with an alternative strategy

State machine:

```
idle → listening → thinking → acting → speaking → idle
      ↑               ↓
      |               |
      └── (user interrupts) ─┘
```

Each transition triggers a feedback sound.

Concurrency: Two parallel async loops:

1. `_client_receive_loop()` – messages from client (audio, page state, action results, tab events, page load events)
2. `_gemini_receive_loop()` – events from Gemini (audio output, tool calls, text, interruptions)

Pending tool calls tracked in a `dict[str, asyncio.Event]` keyed by `tool_call_id`.

[NEW] backend/core/gemini_live.py

Wrapper around Google's Gemini Live API (multimodal streaming WebSocket).

What Gemini Live provides in one connection:

- Real-time speech-to-text
- Reasoning
- Text-to-speech (streaming audio output)
- Function calling (triggers browser tools mid-conversation)
- Vision (receives screenshots)
- Session memory (conversation context persists)

Interface:

python

```
class GeminiLiveClient:
    @classmethod
    async def connect(cls, tools, system_instruction) -> "GeminiLiveClient"
    async def send_audio(self, pcm16_bytes: bytes) -> None
    async def send_text(self, text: str) -> None
    async def send_tool_result(self, tool_call_id: str, result: dict) -> None
    async def send_image(self, image_b64: str, mime_type: str) -> None
    async def events(self) -> AsyncGenerator[GeminiEvent, None]
    async def close(self) -> None
```

GeminiEvent types: `audio`, `tool_call`, `text`, `interrupted`, `error`

[NEW] backend/core/tools.py

All 25 tool definitions:

python

```
BROWSER_TOOLS = [
    # == Page Understanding (3) ==
    {
        "name": "read_page",
        "description": "Get the page's full state: URL, title, accessibility tree, heading structure, form states, main text, math formulas. Also detects and reports cookie banners, modals, and overlays blocking the page.",
    },
    {
        "name": "capture_screenshot",
        "description": "Take a screenshot for visual analysis. Use when accessibility tree isn't enough – layouts, images, charts, complex UI.",
    },
    {
        "name": "get_orientation",
        "description": "Get current context: page title, breadcrumbs, current section/module, course progress, available navigation. Use after every navigation.",
    },

    # == Navigation (3) ==
    {
        "name": "navigate_to",
        "description": "Navigate to a URL.",
        "parameters": {"url": {"type": "string"}},
    },
    {
        "name": "go_back",
        "description": "Go to previous page in browser history.",
    },
    {
        "name": "scroll_page",
        "description": "Scroll the page.",
        "parameters": {
            "direction": {"type": "string", "enum": ["up", "down"]},
            "amount": {"type": "integer", "description": "Pixels. 500 = half screen."},
        },
    },

    # == DOM Interaction (7) ==
    {
        "name": "click_element",
        "description": "Click an element. Provide CSS selector, ARIA label, or visible text. System tries: CSS → aria-label → text match → role+name → placeholder → label.",
        "parameters": {
            "selector": {"type": "string"},
            "description": {"type": "string", "description": "Human-readable description"},
        },
    },
]
```

```

    },
    {
        "name": "type_text",
        "description": "Type text into a form field. Clears existing content first.",
        "parameters": {
            "selector": {"type": "string", "description": "Null for focused element."},
            "text": {"type": "string"},
        },
    },
    {
        "name": "type_rich_text",
        "description": "Type into a rich text editor (TinyMCE, CKEditor, Quill,
contenteditable). Supports optional formatting via keyboard shortcuts.",
        "parameters": {
            "text": {"type": "string"},
            "format": {"type": "string", "enum": ["plain", "bold", "italic",
"heading"], "description": "Optional formatting."},
        },
    },
    {
        "name": "select_option",
        "description": "Select from a dropdown/select element.",
        "parameters": {
            "selector": {"type": "string"},
            "value": {"type": "string"},
        },
    },
    {
        "name": "clear_field",
        "description": "Clear a text input or textarea.",
        "parameters": {"selector": {"type": "string"}},
    },
    {
        "name": "press_key",
        "description": "Press a keyboard key.",
        "parameters": {"key": {"type": "string", "description": "Enter, Tab, Escape,
ArrowDown, etc."}},
    },
    {
        "name": "dismiss_overlay",
        "description": "Close a modal, popup, cookie banner, or overlay blocking the
page. Looks for close/accept/dismiss buttons, X buttons, Escape key, or backdrop
click.",
    },
    # === Keyboard Navigation Fallback (1) ===
    {
        "name": "keyboard_navigate",
        "description": "Press keyboard keys in sequence to navigate. Use when DOM click
doesn't work – cross-origin iframes (payment forms), custom widgets, date pickers. Tab
between fields, Arrow keys in menus, Enter to activate, Space for checkboxes.",
        "parameters": {
            "keys": {"type": "array", "items": {"type": "string", "description": "Keys
in sequence: Tab, Shift+Tab, Enter, Space, Escape, ArrowUp, ArrowDown, etc."},

```

```

    },
  },

  # == Video Control (6) ==
  {
    "name": "play_video",
    "description": "Play the video. Works with native HTML5, Video.js, and via
keyboard Space/K for YouTube/Vimeo iframes.",
  },
  {
    "name": "pause_video",
    "description": "Pause the video.",
  },
  {
    "name": "seek_video",
    "description": "Jump to a time in the video.",
    "parameters": {"seconds": {"type": "number"}}},
  },
  {
    "name": "set_video_speed",
    "description": "Change playback speed.",
    "parameters": {"rate": {"type": "number", "description": "0.5, 1.0, 1.25, 1.5,
2.0"}}},
  },
  {
    "name": "get_video_status",
    "description": "Get video state: playing/paused, current time, duration,
speed.",
  },
  {
    "name": "read_transcript",
    "description": "Extract video transcript. Checks: track elements (WebVTT),
platform panels (Coursera, Udemy, YouTube, edX, Moodle), ARIA transcript regions,
downloadable transcript links.",
  },

  # == Quiz (5) ==
  {
    "name": "get_quiz_status",
    "description": "Get quiz state: total questions, answered count, current
question, timer remaining, submitted status.",
  },
  {
    "name": "read_question",
    "description": "Read a specific question and all options.",
    "parameters": {"number": {"type": "integer", "description": "1-indexed. Omit
for current."}}},
  },
  {
    "name": "select_answer",
    "description": "Select an answer. Works for radio buttons, checkboxes, and text
inputs.",
    "parameters": {"value": {"type": "string", "description": "Answer letter
(A/B/C/D) or text."}}},
  },

```


[NEW] backend/core/tool_executor.py

Routes Gemini's tool calls to the correct client WebSocket message:

- `read_page` / `get_orientation` → `request_page_state`
- `capture_screenshot` → `request_screenshot`
- `click_element` / `type_text` / `type_rich_text` / `select_option` / `clear_field` / `press_key` / `navigate_to` / `go_back` / `scroll_page` / `dismiss_overlay` → `execute_action`
- `keyboard_navigate` → `execute_keyboard_nav`
- `play_video` / `pause_video` / `seek_video` / `set_video_speed` / `get_video_status` / `read_transcript` → `execute_video_action`
- `get_quiz_status` / `read_question` / `select_answer` / `submit_quiz` / `get_quiz_timer` → `execute_quiz_action`

Each call stores a pending event with `tool_call_id`, sends the command, and awaits the result with timeout (`ACTION_TIMEOUT_SECONDS`). Returns error to Gemini on timeout so it can retry.

[NEW] backend/core/prompts.py

10 behavioral rule sections:

```
python
```

```
SYSTEM_PROMPT = """
```

```
You are TernConnect, a voice-based AI assistant that gives blind and  
visually impaired users full control over web platforms.
```

SECTION 1: YOUR ROLE

You ARE the user's eyes and hands. They speak, you see the page and act for them. Everything you say is spoken aloud – write for the ear.

SECTION 2: ORIENTATION (CRITICAL)

After EVERY navigation or major page change, automatically provide:

1. WHERE: Page title + context ("Lesson 3 of Python for Everybody")
2. WHAT: What's on this page ("A 15-minute video and a practice quiz")
3. OPTIONS: Top 2-3 actions ("Play the video, read the transcript, or skip to the quiz")

NEVER leave the user on a new page without orienting them.

Always-available commands (respond immediately):

- "Where am I?" → Full orientation
- "What can I do?" → Available actions
- "Go back" → Previous page
- "Help" → Usage guide

SECTION 3: SPEAKING STYLE

- Concise and conversational – user is listening, not reading
- Numbers: "four point eight" not "4.8"
- Never say "I can see" – say "the page shows" or "there is"
- Positional language: "first item", "at the top", "third option"
- Never read URLs, selectors, HTML, or code aloud
- Never describe your own tool calls

SECTION 4: ACTION NARRATION

- Before acting: "Clicking the search button now"
- After acting: "Search results loaded. Found 15 courses."
- On error: "That button didn't respond. Trying another way."
- Proactive: "I notice this course requires a subscription."
- Confirm before destructive actions: "Submit the quiz? 2 unanswered."

SECTION 5: LMS EXPERTISE

You deeply understand LMS patterns:

- Course catalogs, enrollment flows, progress tracking
- Video players, transcripts, downloadable resources
- Quizzes: multiple choice, fill-in-blank, matching, ordering
- Discussion forums, peer review, assignments
- Platforms: Coursera, Udemy, edX, Moodle, Canvas, Khan Academy, Blackboard

SECTION 6: VIDEO HANDLING

Videos are ~50% of LMS content. The user CANNOT watch them.

1. Mention video + duration when a page has one
2. Offer: "Play the video, or read the transcript?"
3. Read transcripts naturally – paraphrase where appropriate
4. "Skip to [topic]" → search transcript for that section
5. No transcript? Take periodic screenshots and describe content
6. Support: pause, seek, speed changes

SECTION 7: QUIZ PROTOCOL

1. Announce: "10 questions. Time limit: 15 minutes." (or "No time limit")
2. Read: question number, text, ALL options
3. Never answer for the user unless explicitly asked
4. Confirm: "Selecting B: $O(\log n)$. Next question."
5. Progress updates every 3–4 questions
6. Before submit: "Answered all 10. Ready to submit?"
7. After submit: score + incorrect question numbers

TIMED QUIZ: Announce remaining time at 50%, 25%, 5min, 2min, 1min, 30sec.
"2 minutes left, 3 unanswered. Want me to read faster?"

DRAG-AND-DROP / MATCHING / ORDERING:

1. Describe what needs to be matched/ordered
2. Ask user to state the correct pairings/order verbally
3. Use keyboard alternatives or programmatic DOM reordering

SECTION 8: TUTOR MODE

You're a learning companion, not just a screen reader.

- "Explain this" → own words, analogies, examples
- After lessons, offer: "Want me to quiz you?"
- Wrong answer + "why?" → explain correct answer with reasoning
- "I don't understand" → try simpler terms, different angle
- Be encouraging: "Good job!", "That's tricky", "Great progress"

SECTION 9: OBSTACLE HANDLING

COOKIE CONSENT / MODALS / POPUPS:

- Detect and dismiss automatically using `dismiss_overlay`

- Always announce: "Closing a cookie banner" / "Dismissing a popup"

CAPTCHA:

- If audio CAPTCHA exists → click audio button, listen, type answer
- If only visual → "There's a verification I can't solve. Try 'Sign in with Google', or ask someone nearby to help with this one step."

OAuth / New Tabs:

- "A login window opened. I'll handle it."
- Complete the flow, then: "Done. Back to [original page]."

Session Expiry:

- If redirected to login unexpectedly: "Your session expired. Logging you back in."
- Re-authenticate and navigate back to previous page

Page Loading:

- During loads: "Page loading..."
- When ready: auto-orient with page description

Payment Forms (Stripe/PayPal iframes):

- Use keyboard_navigate (Tab between fields, type card info)
- "I'll guide you through the payment form field by field."

SECTION 10: CONTENT NARRATION

Code Blocks:

- "Here's a code example in Python"
- Read line by line with indentation: "indented: if x equals 5, colon"
- Long blocks: summarize first, then offer line-by-line
- Spell out ambiguous symbols: "open parenthesis, x, comma, y, close parenthesis"

Mathematical Formulas:

- Read as spoken math: " $x^2 + 2x + 1$ "
- Fractions: "3 over 4"
- Summation: "the sum from i equals 1 to n of x_i "
- Complex formulas: "Let me break this down step by step..."

Images / Diagrams:

- Use capture_screenshot + describe for images without alt text
- Charts: describe trend, key data points, axes
- Diagrams: describe relationships and flow

PDFs / Documents:

- Extract text from browser PDF viewer
 - Or screenshot each page and describe via Gemini vision
- ""

[NEW] backend/api/websocket_handler.py

Handles WebSocket lifecycle for one client:

- Accepts connection
- Creates `ClientSession` (opens Gemini Live session)
- Sends greeting + connection feedback sound
- Runs session until disconnect
- Catches errors, sends error feedback, logs, cleans up

[NEW] backend/audio/feedback.py

Audio feedback state machine:

python

```
class AudioFeedback:
    """
    State transitions → sounds:
    idle → listening:      chime_up.wav
    listening → thinking:  chime_down.wav
    thinking (ongoing):    thinking.wav (looping)
    acting:                tick.wav (per action)
    action success:        success.wav
    action error:          error.wav
    page loading:          whoosh.wav
    page ready:            chime_up.wav (short)
    speaking:              (Gemini voice is playing)
    → idle:                (silence – waiting)
    """

    def get_transition_sounds(self, old_state, new_state) -> list[dict]:
        """Returns sound messages to send to the client."""
```

Loads `.wav` files, base64-encodes, sends via WebSocket. Client plays feedback sounds and Gemini speech on separate audio channels.

[NEW] backend/models/messages.py

Complete WebSocket protocol – 20 message types:

Client → Backend (11 types):

Type	Purpose
<code>audio_chunk</code>	Streaming mic PCM16 audio
<code>page_state</code>	Response to <code>request_page_state</code> – accessibility tree, headings, forms, math content
<code>screenshot</code>	Response to <code>request_screenshot</code> – base64 JPEG
<code>action_result</code>	Result of DOM/keyboard/overlay action – success/error
<code>video_status</code>	Response to <code>get_video_status</code> – playing, time, duration, speed
<code>transcript</code>	Response to <code>read_transcript</code> – full text
<code>quiz_status</code>	Response to <code>get_quiz_status</code> / <code>get_quiz_timer</code>
<code>quiz_question</code>	Response to <code>read_question</code> – text + options
<code>listening_started</code>	User pressed shortcut
<code>listening_stopped</code>	User released shortcut
<code>page_loading</code>	Tab status changed to loading
<code>page_loaded</code>	Tab status changed to complete
<code>tab_changed</code>	New tab/popup opened or closed
<code>session_expired</code>	Unexpected redirect to login page detected
<code>quiz_timer_update</code>	Timer threshold reached (pushed proactively)

Backend → Client (9 types):

Type	Purpose
<code>audio_chunk</code>	Gemini TTS audio to play
<code>feedback_sound</code>	Non-speech audio cue (sound name + loop flag)
<code>speech_text</code>	Text of what Gemini is saying (for ARIA live region)
<code>request_page_state</code>	Ask for accessibility tree
<code>request_screenshot</code>	Ask for screenshot
<code>execute_action</code>	DOM action (click, type, select, clear, press_key, navigate, scroll, dismiss_overlay, type_rich_text)
<code>execute_keyboard_nav</code>	Keyboard key sequence to press
<code>execute_video_action</code>	Video control (play, pause, seek, speed, status, transcript)
<code>execute_quiz_action</code>	Quiz interaction (read_question, select, submit, status, timer)

[NEW] backend/Dockerfile

```
dockerfile

FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

[NEW] backend/requirements.txt

```
fastapi>=0.110.0
uvicorn[standard]>=0.29.0
websockets>=12.0
google-genai>=1.0.0
python-dotenv>=1.0.0
```


[NEW] extension/manifest.json

```
json

{
  "manifest_version": 3,
  "name": "TernConnect",
  "version": "1.0.0",
  "description": "AI accessibility agent – navigate any website by voice",
  "permissions": ["activeTab", "scripting", "offscreen", "storage", "tabs",
"webNavigation"],
  "host_permissions": ["<all_urls>"],
  "background": {
    "service_worker": "background/service-worker.js"
  },
  "content_scripts": [{
    "matches": ["<all_urls>"],
    "all_frames": true,
    "js": [
      "content/page-reader.js",
      "content/dom-controller.js",
      "content/video-handler.js",
      "content/video-controller.js",
      "content/quiz-monitor.js",
      "content/accessibility.js",
      "content/keyboard-navigator.js"
    ],
    "css": ["content/content.css"],
    "run_at": "document_idle"
  }],
  "action": {
    "default_popup": "popup/popup.html",
    "default_icon": { "16": "icons/icon16.png", "48": "icons/icon48.png", "128":
"icons/icon128.png" }
  },
  "commands": {
    "toggle-listening": {
      "suggested_key": { "default": "Ctrl+Shift+Space" },
      "description": "Start/stop voice input"
    }
  }
}
```

Key: `"all_frames": true` – injects into iframes for cross-origin content access.

[NEW] extension/background/service-worker.js

Central router:

- **WebSocket management:** Connects to `wss://api.ternconnect.com/ws`, auto-reconnects with exponential backoff
- **Shortcut handler:** `chrome.commands.onCommand` for `Ctrl+Shift+Space`
- **Message routing:** Backend messages → correct handler (content script, offscreen doc)
- **Tab monitoring:** `chrome.tabs.onUpdated` for page loading/loaded events
- **Popup/tab tracking:** `chrome.tabs.onCreated` / `chrome.tabs.onRemoved` for OAuth popups and new tabs
- **Cross-origin frame injection:** `chrome.webNavigation.onCompleted` → `chrome.scripting.executeScript` to inject into frames that `all_frames` couldn't reach
- **Badge update:** Changes extension icon color based on status (idle=gray, listening=green, thinking=yellow, speaking=blue)

[NEW] extension/content/page-reader.js

Extracts full page state:

```
javascript

class PageReader {
  getPageState() {
    return {
      url: location.href,
      title: document.title,
      accessibility_tree: this._buildAccessibilityTree(document.body, 0),
      headings: this._getHeadingStructure(),
      forms: this._getFormState(),
      interactive_elements: this._getInteractiveElements(),
      main_text: this._getMainTextContent(),
      math_content: this._extractMathContent(),
      overlays: this._detectOverlays(),
      is_pdf: this._isPDFPage(),
    };
  }

  _buildAccessibilityTree(node, depth) {
    // Walks DOM, per element: { role, name, value, selector, interactive, children }
    // Skips hidden/decorative. Max depth 8. Prioritizes ARIA roles.
  }

  _getAccessibleName(node) {
    // aria-label → aria-labelledby → alt → title → placeholder → label[for] →
    textContent
  }

  _buildSelector(node) {
    // Stable selector priority: #id → [data-testid] → [aria-label] → tag[role][name] →
    nth-child
  }

  _extractMathContent() {
    // MathJax: .MathJax elements → extract original LaTeX from data attributes
    // KaTeX: .katex elements → extract LaTeX
    // Returns array of { latex, location_description }
  }

  _detectOverlays() {
    // Cookie banners, modals, popups blocking the page
    // Detects: [role="dialog"], .modal, .cookie-banner, position:fixed overlays
    // Returns: { found, type, close_selector }
  }

  _isPDFPage() {
    // Checks: URL ends in .pdf, embed[type="application/pdf"], Chrome PDF viewer
  }

  // Session expiry detection
}
```


[NEW] extension/content/dom-controller.js

Executes DOM actions with multi-strategy element finding:

javascript

```
class DOMController {
  async executeAction({ action, params }) {
    switch (action) {
      case 'click_element': return this._click(params.selector);
      case 'type_text': return this._type(params.selector, params.text);
      case 'type_rich_text': return this._typeRichText(params.text, params.format);
      case 'select_option': return this._select(params.selector, params.value);
      case 'clear_field': return this._clear(params.selector);
      case 'press_key': return this._pressKey(params.key);
      case 'navigate_to': window.location.href = params.url; return { success:
true };
      case 'go_back': history.back(); return { success: true };
      case 'scroll_page': return this._scroll(params.direction, params.amount);
      case 'dismiss_overlay': return this._dismissOverlay();
    }
  }

  _findElement(selector) {
    // 1. CSS selector
    // 2. [aria-label="..."] or [aria-label*="..."]
    // 3. Text content match (buttons, links)
    // 4. [role="..."] with matching accessible name
    // 5. input[placeholder*="..."]
    // 6. Label text → linked input
    // Returns null if not found
  }

  _click(selector) {
    const el = this._findElement(selector);
    if (!el) return { success: false, error: `Element not found: ${selector}` };
    this._flashHighlight(el);
    el.scrollIntoView({ block: 'center' });
    el.focus();
    el.click();
    return { success: true };
  }

  _type(selector, text) {
    const el = selector ? this._findElement(selector) : document.activeElement;
    if (!el) return { success: false, error: `Input not found: ${selector}` };
    el.focus();
    el.value = '';
    el.value = text;
    el.dispatchEvent(new Event('input', { bubbles: true }));
    el.dispatchEvent(new Event('change', { bubbles: true }));
    return { success: true };
  }
}
```


[NEW] extension/content/video-handler.js

Video detection and caption/transcript extraction:

```
javascript

class VideoHandler {
  detectVideo() {
    // Returns: { found, playerType, hasTranscript, duration }
    // Checks: <video>, .vjs-tech, YouTube iframe, Vimeo iframe, Wistia, platform-
specific
  }

  extractTranscript() {
    // 1. <track kind="captions"/>"subtitles"> → parse WebVTT
    // 2. Coursera: .rc-Transcript
    // 3. Udemy: [data-purpose="transcript-cue-container"]
    // 4. YouTube: ytd-transcript-segment-renderer
    // 5. edX: .video-transcript
    // 6. Moodle: .mediaplugin-transcript
    // 7. Generic: [role="region"] with "transcript" in label
    // 8. Downloadable transcript link (last resort)
    // Returns: { text, segments: [{time, text}], source }
  }
}
```

[NEW] extension/content/video-controller.js

Universal video controls:

```
javascript

class VideoController {
  play()           // .play() on <video> or click play button or keyboard Space
  pause()          // .pause() or click pause or keyboard Space
  seek(seconds)    // .currentTime = seconds or player API
  setSpeed(rate)   // .playbackRate = rate or find speed control UI
  getStatus()      // { playing, currentTime, duration, speed }

  _getVideoElement() {
    // Native <video> → .vjs-tech → search inside iframes
  }
}
```

For cross-origin iframe videos (YouTube/Vimeo): uses `keyboard_navigate` with player keyboard shortcuts (Space for play/pause, J/L for seek, Shift+</> for speed).

[NEW] extension/content/quiz-monitor.js

Quiz state tracking and proactive timer monitoring:

```
javascript

class QuizMonitor {
  getQuizStatus()           // total, answered, current, timer_remaining, submitted
  readQuestion(number)      // number, text, options[{{label, text, selected}}, type
  selectAnswer(value)       // Click radio/checkbox or fill text input
  submitQuiz()              // Click submit button

  // Drag-and-drop support
  reorderElement(sourceSelector, targetPosition) {
    // Dispatch DragEvent programmatically
    // Or use keyboard alternatives if available
  }

  // Timer monitoring – pushes updates at key thresholds
  watchTimer() {
    // MutationObserver on timer element
    // Pushes quiz_timer_update at: 50%, 25%, 5min, 2min, 1min, 30sec
  }
}
```


[NEW] extension/content/accessibility.js

Screen reader bridge:

```
javascript

class AccessibilityBridge {
  constructor() {
    // Hidden ARIA live region for screen reader announcements
    this.liveRegion = document.createElement('div');
    this.liveRegion.setAttribute('role', 'status');
    this.liveRegion.setAttribute('aria-live', 'polite');
    this.liveRegion.setAttribute('aria-atomic', 'true');
    this.liveRegion.style.cssText = 'position:absolute;left:-9999px;';
    document.body.appendChild(this.liveRegion);
  }

  announce(text) {
    // Announce to screen readers alongside TernConnect TTS
    this.liveRegion.textContent = '';
    requestAnimationFrame(() => { this.liveRegion.textContent = text; });
  }

  detectScreenReader() {
    // Heuristics: forced-colors, prefers-reduced-motion, known extensions
  }
}
```

[NEW] extension/content/keyboard-navigator.js

Fallback for elements unreachable by DOM selectors:

javascript

```
class KeyboardNavigator {
  async pressKeys(keys) {
    // Presses each key in sequence with small delays
    // Supports: Tab, Shift+Tab, Enter, Space, Escape, ArrowUp/Down/Left/Right,
    character keys
    // Uses KeyboardEvent dispatch on document.activeElement
    for (const key of keys) {
      await this._pressKey(key);
      await this._sleep(100); // Small delay between keys
    }
    return { success: true };
  }

  _pressKey(key) {
    // Parse modifiers: "Shift+Tab" → { key: "Tab", shiftKey: true }
    // Dispatch keydown + keyup events
  }
}
```

[NEW] extension/content/content.css

CSS

```
.ternconnect-highlight {
  outline: 3px solid #4a90d9 !important;
  outline-offset: 2px !important;
  transition: outline-color 0.3s ease;
}

.ternconnect-sr-announcer {
  position: absolute;
  left: -9999px;
  width: 1px;
  height: 1px;
  overflow: hidden;
}
```

[NEW] extension/offscreen/audio-handler.js

All audio I/O (Chrome MV3 requires offscreen document for Web Audio):

- **Mic capture:** `getUserMedia()` → AudioWorklet → PCM16 16kHz mono → stream to service worker
- **TTS playback:** Decode base64 audio from backend → play via AudioContext (channel 1)
- **Feedback sounds:** Play on separate AudioContext channel so they don't clip TTS
- **Interruption:** Stop TTS when user starts speaking (service worker signals `listening_started`)
- **Volume control:** Adjustable gain node

[NEW] extension/popup/ (popup.html + popup.js + popup.css)

Screen-reader-accessible settings:

- Connection status: "Connected" / "Disconnected" with color indicator
- Backend URL field (default: `wss://api.ternconnect.com/ws`)
- Shortcut key display
- Toggle: TernConnect TTS on/off (screen reader only mode)
- Narration verbosity: brief / normal / detailed
- All elements have ARIA labels, roles, focus management, high-contrast colors

Deployment

```
dockerfile

# Build + deploy
docker build -t ternconnect-backend ./backend
docker push registry/ternconnect-backend

# Run on Railway / Fly.io / AWS ECS / GCP Cloud Run
# All support WebSocket connections
```

Clients connect to `wss://api.ternconnect.com/ws` .

Verification Plan

Backend

```
bash

cd ternconnect/backend
pip install -r requirements.txt
uvicorn main:app --reload --port 8000
curl http://localhost:8000/health # → {"status": "ok"}
```

Extension

1. chrome://extensions → Developer mode → Load unpacked → select extension/
2. Open popup → verify "Connected"
3. Navigate to coursera.org
4. Ctrl+Shift+Space → speak → verify audio response

End-to-End Scenarios

- | | |
|----------------------|--|
| 1. Course Discovery: | "Find python courses on Coursera" |
| 2. Cookie Dismissal: | Cookie banner auto-dismissed, announced |
| 3. Account Creation: | "Sign me up" → form fill → CAPTCHA handling |
| 4. OAuth Login: | "Sign in with Google" → popup handled → back to main |
| 5. Enrollment: | "Enroll in the first course" → payment form via keyboard nav |
| 6. Video Lesson: | "Read the transcript" → transcript extracted and narrated |
| 7. Quiz (untimed): | Full question→answer→submit→score flow |
| 8. Quiz (timed): | Timer announcements at thresholds |
| 9. Discussion Post: | "Post a comment" → rich text editor → submit |
| 10. Session Expiry: | Session timeout → auto-relogin → navigate back |
| 11. Error Recovery: | Bad selector → retry with alternative → success |
| 12. Math Content: | LaTeX formula read as spoken math |

Phases

Phase 1 – Core + Chrome Extension (BUILD NOW)

- ☒ FastAPI backend with Gemini Live API
- ☒ 25 Gemini tools (page, nav, DOM, video, quiz, keyboard, overlay)
- ☒ System prompt (10 sections: orientation, tutor, quiz, obstacles, narration)
- ☒ Audio feedback engine (7 sounds)
- ☒ WebSocket protocol (20 message types)
- ☒ Chrome extension: 8 content scripts + offscreen audio + popup
- ☒ Cross-origin iframe support
- ☒ Screen reader bridge (ARIA)
- ☒ Keyboard navigation fallback
- ☒ Cookie/modal auto-dismissal
- ☒ Session expiry detection
- ☒ Dockerfile for deployment

Phase 2 – LinkedIn + Polish

- ☐ LinkedIn tools and prompts (feed, posts, jobs, messaging)
- ☐ User accounts + auth on backend
- ☐ ElevenLabs premium voice option
- ☐ Wake word detection ("Hey Tern")
- ☐ Usage analytics

Phase 3 – Desktop App

- ☐ Electron wrapper
- ☐ System tray + global hotkeys
- ☐ Mac + Windows installers

Phase 4 – Mobile App

- ☐ React Native voice client
- ☐ Server-side Playwright for mobile browsing
- ☐ VoiceOver / TalkBack integration